

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра информационных систем и технологий

ОТЧЕТ ПО ПРАКТИКЕ

Вид практики производственная
(учебная, производственная)

Тип практики научно-исследовательская работа
(в соответствии с ОПОП ВО)

Сроки прохождения практики: с 01.10.2025 по 30.12.2025
(в соответствии с календарным учебным графиком)

по направлению подготовки 09.03.01 Информатика и вычислительная техника
(уровень бакалавриата)
направленность (профиль) «Информационные системы»

Студент группы № 6404-090301D _____ М.П. Федулов

Руководитель практики,

доцент кафедры ИСТ, к.т.н. _____ Я.В. Соловьева

Дата сдачи 30.12.2025

Дата защиты 30.12.2025

Оценка _____

Самара 2025

СОДЕРЖАНИЕ

Задание(я) для выполнения определенных видов работ, связанных с будущей профессиональной деятельностью (сбор и анализ данных и материалов, проведение исследований)

1 Анализ современных инструментальных средств разработки программного обеспечения для информационных систем, обоснование выбора используемой среды разработки программного обеспечения. Полное описание алгоритмов и программного обеспечения

2 Анализ современных технологий разработки компонентов системных программных продуктов, обоснование выбора используемой технологии. Описание схем основных алгоритмов и их программной реализации

Заключение

Список использованных источников

Приложение А

Приложение Б

Приложение В

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра информационных систем и технологий

**Задание(я) для выполнения определенных видов работ, связанных с
будущей профессиональной деятельностью (сбор и анализ данных и
материалов, проведение исследований)**

Студенту группы 6404-090301D

Федулову Михаилу Павловичу

Направление на практику оформлено приказом по университету
от 29.09.2025 г. № 472-ПР

на кафедру информационных систем и технологий

(наименование профильной организации или структурного подразделения университета)

Тема НИР: «Исследование подходов разработки, взаимодействия и сборки
микрофронтенд приложений»

Планируемые результаты освоения образовательной программы (компетенции)	Планируемые результаты практики	Содержание задания
ПК-1 Способен осуществлять проведение научно- исследовательских и опытно- конструкторских разработок по отдельным разделам темы. ПК 1.3 Демонстрирует способность понимать, совершенствовать и	Знать: современные инструментальные средства разработки программ. Уметь: применять современные инструментальные средства для разработки программ. Владеть: навыками разработки алгоритмов и программ.	Провести анализ современных инструментальных средств разработки программного обеспечения для информационных систем. Сделать обоснование выбора используемой среды разработки программного обеспечения.

применять современный инструментарий в ходе исследований в рамках профессиональной деятельности		Разработать алгоритмы автоматизированной системы по отслеживанию выполнения задач в компании. Сделать описание основных алгоритмов автоматизированной системы по отслеживанию выполнения задач в компании.
ПК-5 Способен осуществлять разработку компонентов системных программных продуктов. ПК-5.1 Разрабатывает драйвера устройств, компиляторы, загрузчики, сборщики, системные утилиты.	Знать: современные технологии разработки компонентов системных программных продуктов. Уметь: применять современные технологии разработки компонентов системных программных продуктов. Владеть: навыками разработки алгоритмов и программ.	Провести анализ современных технологий разработки компонентов системных программных продуктов. Сделать обоснование выбора используемой технологии. Разработать основные алгоритмы и программное обеспечение компонентов системных программных продуктов автоматизированной системы по отслеживанию выполнения задач в компании. Сделать описание схем основных алгоритмов и программного обеспечения автоматизированной системы по отслеживанию выполнения задач в компании.

Дата выдачи задания 01.10.2025.

Срок представления на кафедру отчета о практике 30.12.2025.

Руководитель НИР,

доцент кафедры ИСТ, к.т.н. _____ Я.В. Соловьева
(подпись)

Задание принял к исполнению

студент группы № 6404-090301D _____ М.П. Федулов
(подпись)

1 Анализ современных инструментальных средств разработки программного обеспечения для информационных систем, обоснование выбора используемой среды разработки программного обеспечения. Полное описание алгоритмов и программного обеспечения

1.1 Выбор среды разработки и инструментов

1.1.1 Среда для выполнения и свертки приложения

- Docker – это платформа контейнеризации. Она позволяет упаковать приложение вместе со всеми зависимостями в изолированную среду - контейнер. В отличие от виртуальной машины, которая имитирует полноценные компьютеры со своей ОС, драйверами и файловой системой. Это занимает большое количество места и требует отдельного обслуживания. Docker использует контейнеризацию на уровне ОС, что позволяет не поднимать отдельную ОС под каждое приложение, а создает изолированные контейнеры, которые делят ядро хостовой системы, но имеют собственное пространство процессов, файлов и сетей [1].

- Виртуальные машины (ВМ) - это абстрактные вычислительные экземпляры, созданные программой, запущенной на другом компьютере, которые физически не существуют, но ведут себя как реальный компьютер. виртуальная машина работает на программном обеспечении, а не на оборудовании, другими словами, программное обеспечение создает "воображаемый" компьютер, который ведет себя как физический компьютер, пользователь создает компьютер внутри другого компьютера. Несколько виртуальных машин могут работать параллельно на одном и том же основном устройстве независимо. [2]

Docker выбран из-за его небольшого веса и высокой скорости развертывания.

1.1.2 Языки программирования

- Python - высокоуровневый язык программирования. Благодаря ясному синтаксису его код понятен и легко читается. Python часто называют

«исполнимым псевдокодом»: на нём можно писать код быстрее, чем на других языках [3].

- C++ - представляет высокоуровневый компилируемый язык программирования общего назначения со статической типизацией, который подходит для создания самых различных приложений [4].

- C# - это кроссплатформенный язык общего назначения, который делает разработчиков продуктивным при написании высокопроизводительного кода. С миллионами разработчиков C# является самым популярным языком .NET. C# имеет широкую поддержку в экосистеме и для всех рабочих нагрузок .NET. Основанный на объектно-ориентированных принципах, он включает множество функций из других парадигм, включая функциональное программирование. Низкоуровневые функции поддерживают сценарии высокой эффективности без написания небезопасного кода. Большая часть среды выполнения и библиотек .NET написана на C#, и прогресс в C# часто используется для всех разработчиков .NET [5].

- Java — это объектно-ориентированный язык программирования, разработанный компанией Sun Microsystems (ныне принадлежит Oracle) и впервые выпущенный в 1995 году. Основные принципы Java включают кроссплатформенность, надёжность, безопасность и простоту. Ключевая особенность языка — «пиши один раз, запускай где угодно» (Write Once, Run Anywhere), благодаря использованию виртуальной машины Java (JVM), которая интерпретирует байт-код, независимо от операционной системы. Java поддерживает автоматическую сборку мусора, исключения, многопоточность и богатую стандартную библиотеку. Широко применяется в разработке корпоративных приложений, Android-приложений, веб-сервисов, а также в научных и образовательных целях. Несмотря на появление более современных языков, Java остаётся одним из самых популярных и востребованных языков программирования в мире [6].

Для выполнения задания используется язык программирования Java, из-за его надежности, безопасности и возможности адаптации к крупным компаниям.

1.1.3 Фреймворки для разработки фронтенда

React — один из самых популярных фронтенд-фреймворков (точнее, библиотек), разработанный Facebook. Он основан на компонентном подходе и использует виртуальный DOM для эффективного обновления интерфейса. React позволяет создавать интерактивные и производительные пользовательские интерфейсы, особенно для одностраничных приложений (SPA). Благодаря огромному сообществу, богатой экосистеме (например, React Router, Redux) и поддержке TypeScript, он широко применяется в индустрии — от стартапов до крупных корпораций [7].

Angular — полноценный фреймворк от Google, построенный на TypeScript. Он предоставляет всё необходимое «из коробки»: маршрутизацию, управление состоянием, HTTP-клиент, инструменты для тестирования и компиляции. Angular использует двухстороннее связывание данных и мощную систему модулей, что делает его подходящим для крупных и сложных корпоративных приложений. Однако из-за высокой сложности и объёма возможностей может быть менее удобен для небольших проектов или начинающих разработчиков [8].

Vue.js — прогрессивный фреймворк, созданный Эваном Ю. Он сочетает простоту внедрения с возможностью масштабирования до уровня сложного SPA. Vue предлагает интуитивно понятный синтаксис, реактивную систему управления данными и гибкую архитектуру. Он легче в освоении, чем Angular, и при этом предоставляет больше структуры «из коробки», чем React. Vue особенно популярен среди независимых разработчиков и небольших команд благодаря своей лёгкости, читаемости кода и хорошей документации [9].

Svelte — относительно новый, но стремительно набирающий популярность фреймворк, который отличается принципиально иным подходом: вместо виртуального DOM он компилирует компоненты в чистый JavaScript на

этапе сборки. Это делает итоговые приложения быстрее и легче. Svelte требует меньше шаблонного кода, проще в освоении и отлично подходит для проектов, где важна производительность и малый размер бандла. Хотя его экосистема пока меньше, чем у React или Vue, он активно развивается и привлекает внимание разработчиков, ищущих современные и эффективные решения [10].

Наиболее оптимальным выбором будет фреймворк React, из-за его гибкости и простоты интеграции с любым бекендом, большого количества готовых компонентов и библиотек.

1.2 Описание алгоритмов и программного обеспечения

1.2.1 Алгоритм по отображению всех задач на главной странице

Этот модуль отвечает за отображение всех задач сотрудника конкретного отдела. Для начала запрашивается список задач для выбранного отдела с помощью запроса в базу данных. Для этого используется запрос SELECT.

После получения данных необходимо обработать их и отобразить на странице. Далее уже сотрудник выбирает задачу, помечает ее как свою и приступает к выполнению. После выполнения задания необходимо завершить её в менеджере.

В приложении А представлен фрагмент кода для отображения всех записей. На рисунке 1 представлена граф-схема данного алгоритма.

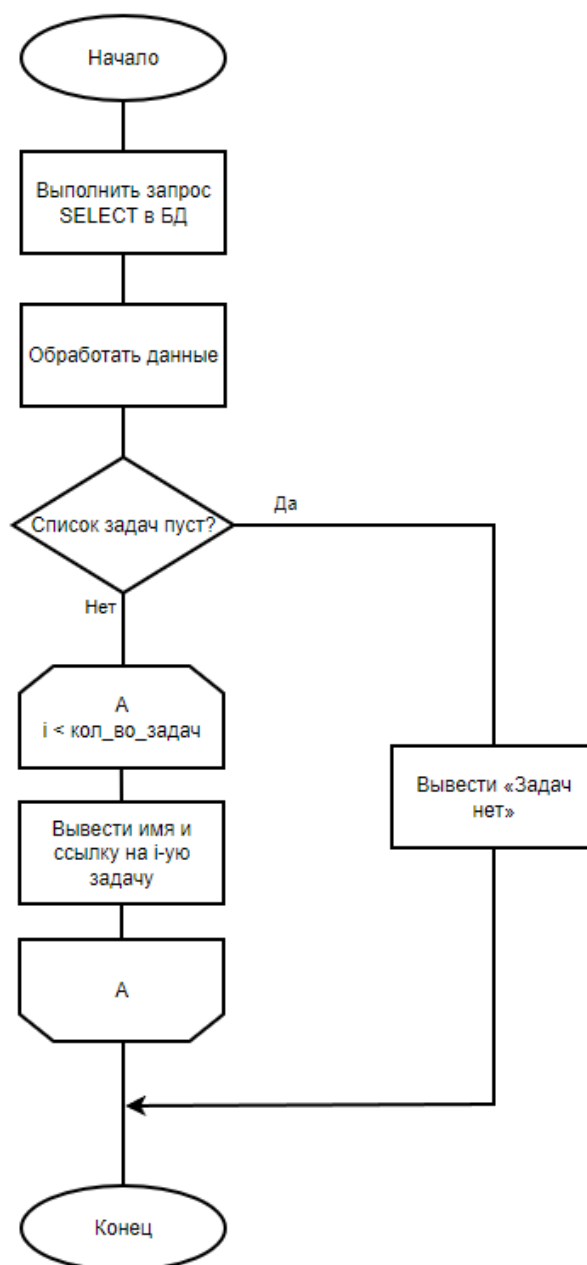


Рисунок 1 – Граф-схема алгоритма по выводу всех доступных задач

1.2.2 Алгоритм добавления задачи

Этот модуль отвечает за формирование задач для сотрудников. Для начала пользователю необходимо заполнить всю необходимую информацию о задаче, а именно: название, описание, срочность, сроки выполнения, исполняющий отдел. Далее пользователь подтверждает действие, проверяет все вводные и получает подтверждение или отказ в формировании задачи.

В приложении Б представлен фрагмент кода данного алгоритма. На рисунке 2 представлена граф-схема алгоритма.

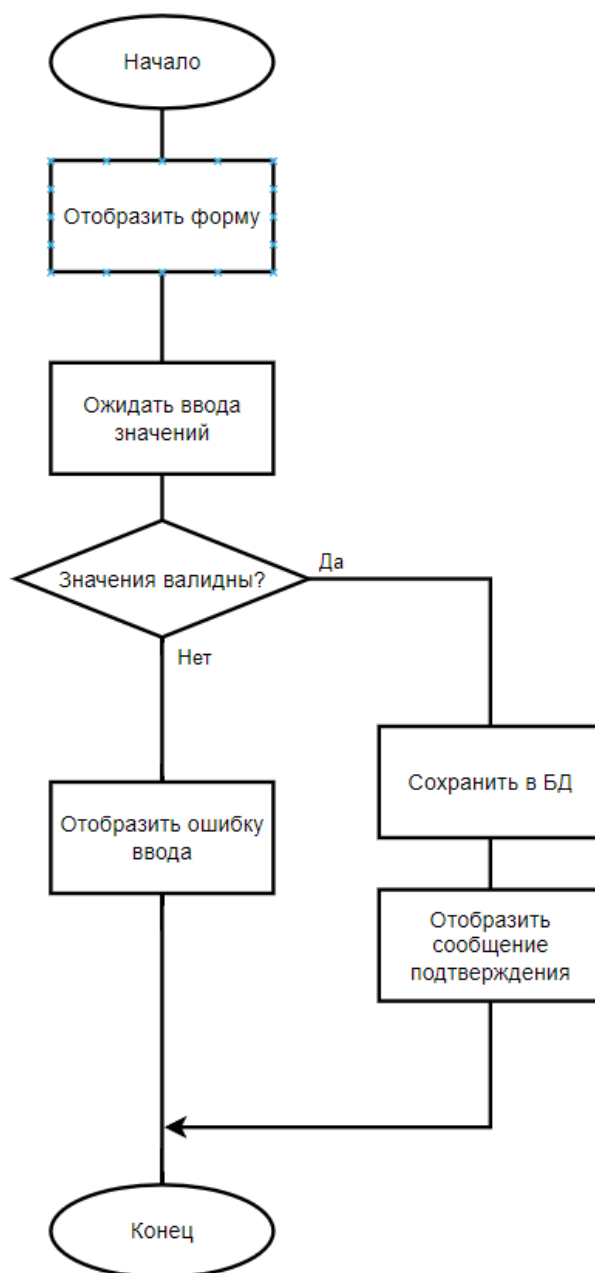


Рисунок 2 – Граф-схема алгоритма по добавлению задачи

1.2.3 Алгоритм авторизации пользователя

Этот модуль отвечает за авторизацию в приложении и последующие операции с задачами. Пользователь вводит свои данные в форму и получает доступ к своей странице с задачами для его отдела.

В приложении В представлен фрагмент кода данного алгоритма. На рисунке 3 представлена граф-схема алгоритма.

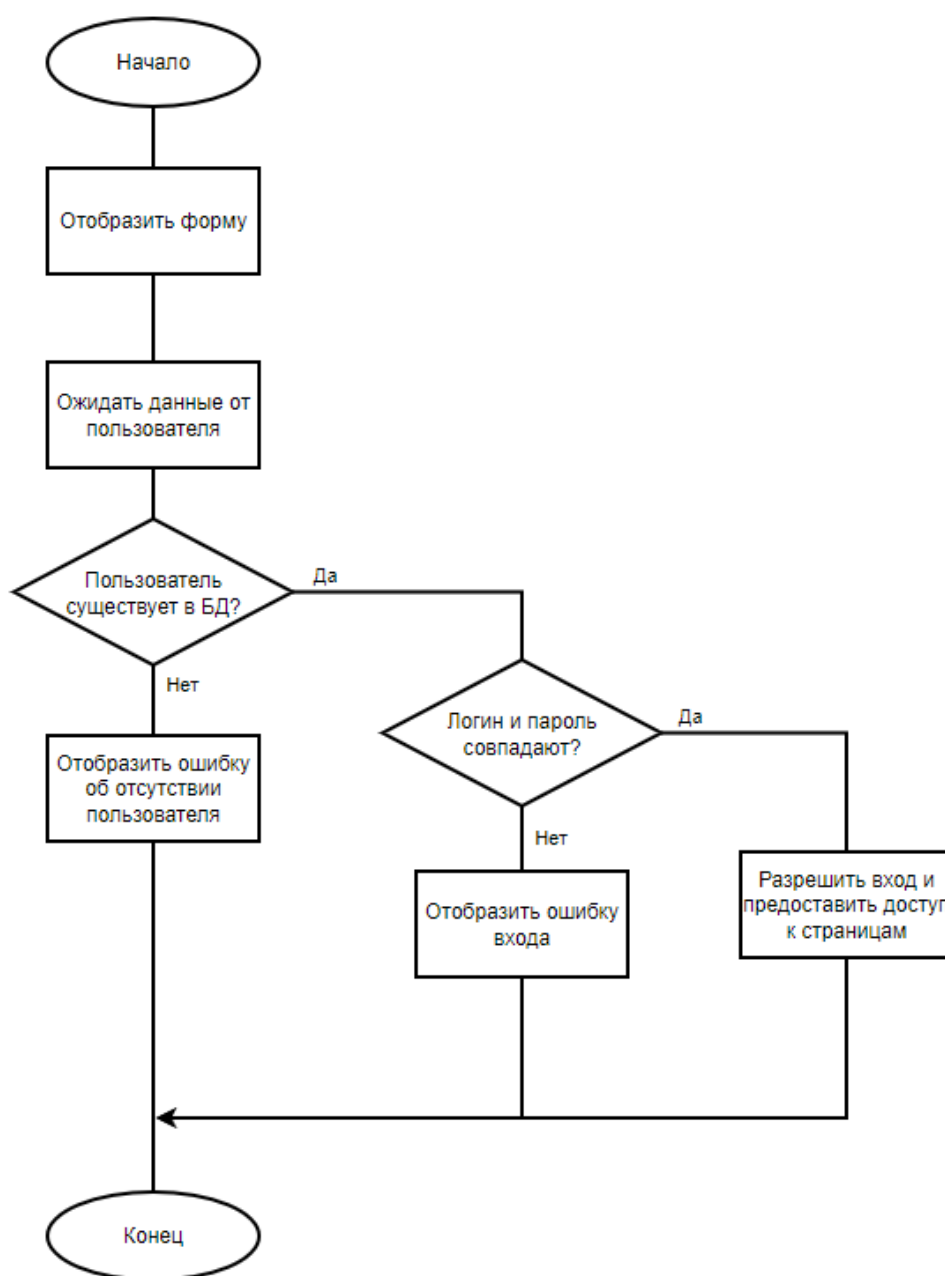


Рисунок 3 – Граф-схема алгоритма авторизации пользователя на портале

2 Анализ современных технологий разработки компонентов системных программных продуктов, обоснование выбора используемой технологии. Описание схем основных алгоритмов и их программной реализации

2.1 Анализ и обоснование технологического стека

При разработке веб-приложения выбор технологий встал в пользу следующих технологий, определяющих архитектуру продукта:

- Технология серверной разработки (Java + Spring Boot)

Java обеспечивает высокую стабильность, строгую типизацию и автоматическое управление памятью, что критично для систем, обрабатывающих важные бизнес-данные. Фреймворк Spring Boot предоставляет готовую инфраструктуру для создания RESTful-сервисов, встроенную поддержку безопасности (Spring Security), управление транзакциями и интеграцию с реляционными базами данных (через Spring Data JPA). Это позволяет быстро и эффективно реализовать бизнес-логику.

- Технология клиентской разработки (React)

React позволяет создавать компонентные, производительные и легко поддерживаемые пользовательские интерфейсы. Его архитектура на основе состояния и однонаправленного потока данных упрощает управление сложностью даже при расширении функционала (например, добавлении уведомлений, дашбордов или фильтров). Интеграция с REST API, реализованным на Spring Boot, осуществляется через стандартные HTTP-запросы, что обеспечивает чёткое разделение клиентской и серверной логики.

- Технология контейнеризации (Docker)

Для обеспечения идентичного поведения приложения во всех средах — от разработки до продакшена — используется Docker. Контейнеризация позволяет упаковать Java-приложение, базу данных, фронтенд и все зависимости в изолированные образы. Это устраняет проблему привязки к одной конфигурации системы, упрощает развёртывание, масштабирование и управление версиями. Сборка и запуск всего стека возможны одной командой

через Docker Compose, что особенно ценно при частых обновлениях и интеграции в CI/CD-конвейеры.

2.2 Описание схем основных алгоритмов и их программной реализации

2.2.1 Алгоритм инициализации и управления задачами

Алгоритм отвечает за создание объекта задачи, валидацию входных данных и сохранение в базе. Реализован с использованием объектно-ориентированного подхода и шаблонов проектирования. На стороне сервера задача представляется как JPA-сущность с необходимыми полями.

Код инициализации объекта задачи и сохранение в БД:

```
Task task = new Task();
task.setTitle(request.getTitle());
task.setDescription(request.getDescription());
task.setUrgency(Urgency.valueOf(request.getUrgency().toUpperCase()));
task.setDeadline(LocalDate.parse(request.getDeadline()));
task.setAssignedDepartment(departmentService.findById(request.getDepartmentId()));
task.setStatus(TaskStatus.CREATED);
task.setCreatedAt(LocalDateTime.now());
taskRepository.save(task);
```

2.2.2 Алгоритм аутентификации и авторизации

Алгоритм обеспечивает безопасный вход пользователя и контроль доступа к ресурсам. Реализован с использованием Spring Security и сессионной аутентификации. При отправке формы авторизации система проверяет учётные данные в хранилище и создаёт аутентифицированный контекст.

Код с аутентификацией пользователя:

```
Authentication auth = authenticationManager.authenticate(
    new UsernamePasswordAuthenticationToken(username, password)
);
```

```
SecurityContextHolder.getContext().setAuthentication(auth);
```

2.2.3 Алгоритм отображения и фильтрации списка задач

Алгоритм реализует загрузку задач с сервера и их отображение в интерфейсе. На фронтенде используется React-компонент, который при монтировании выполняет HTTP-запрос к API и сохраняет данные в состояние.

Код клиентской реализации:

```
const [tasks, setTasks] = useState([]);
useEffect(() => {
  fetch('/api/tasks')
    .then(res => res.json())
    .then(data => setTasks(data))
    .catch(err => console.error('Ошибка загрузки задач:', err));
}, []);

return (
  <ul>
    {tasks.map(task => (
      <li key={task.id}>
        {task.title} — {task.status} — {task.assignedDepartment.name}
      </li>
    ))}
  </ul>
);
```

ЗАКЛЮЧЕНИЕ

В процессе выполнения научно-исследовательской работы были освоены индикаторы ПК-1.3 и ПК-5.1 компетенций ПК-1, ПК-5 и решены все поставленные задачи:

- был проведен анализ современных инструментальных средств разработки программного обеспечения для информационных систем;

- было сделано обоснование выбора используемой среды разработки программного обеспечения автоматизированной системы по отслеживанию выполнения задач в компании;

- были разработаны основные алгоритмы;

- была разработана программная реализация основных алгоритмов;

- было сделано описание основных алгоритмов и программного обеспечения автоматизированной системы по отслеживанию выполнения задач в компании;

- был проведен анализ современных технологий разработки компонентов системных программных продуктов;

- было сделано обоснование выбора используемой технологии;

- были разработаны основные алгоритмы;

- была разработана программная реализация основных алгоритмов;

- было сделано описание схем основных алгоритмов и программного обеспечения автоматизированной системы по отслеживанию выполнения задач в компании.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Петров, А. Docker для начинающих: что это такое и как пользоваться [Электронный ресурс] // URL: <https://habr.com/ru/companies/netologyru/articles/967546/> (дата обращения 25.12.2025)
2. Иванов, Б. Знакомство с виртуальными машинами [Электронный ресурс] // URL: <https://habr.com/ru/companies/timeweb/articles/665786/> (дата обращения 25.12.2025)
3. Сидоров, В. Какой язык программирования выбрать? Обзор Python [Электронный ресурс] // URL: <https://habr.com/ru/companies/kryptonite/articles/807345/> (дата обращения 25.12.2025)
4. Metanit.com. C++: руководство для начинающих [Электронный ресурс] // URL: <https://metanit.com/cpp/tutorial/1.1.php> (дата обращения 25.12.2025)
5. Microsoft Learn. Краткий обзор языка C# [Электронный ресурс] // URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/tour-of-csharp/overview> (дата обращения 25.12.2025)
6. Что такое Java и каково ее применение? [Электронный ресурс] // URL: https://www.java.com/ru/download/help/whatis_java.html (дата обращения: 25.12.2025)
7. Основы React: всё, что нужно знать для начала работы [Электронный ресурс] // URL: <http://habr.com/ru/companies/ruvds/articles/343022> (дата обращения: 25.12.2025)
8. Введение в Angular [Электронный ресурс] // URL: <https://sky.pro/wiki/javascript/vvedenie-v-angular/> (дата обращения: 25.12.2025)
9. Всё, что нужно для начала работы с Vue.js [Электронный ресурс] // URL: <https://habr.com/ru/companies/ruvds/articles/458324/> (дата обращения: 25.12.2025)

10. Что такое Svelte и почему веб-разработчикам стоит обратить на него
внимание [Электронный ресурс] // URL:
<https://timeweb.com/ru/blog/authors/space-police/articles/что-такое-svelte/> (дата
обращения: 25.12.2025)

ПРИЛОЖЕНИЕ А

Фрагмент кода для вывода списка доступных задач

```
function TaskList({ tasksList }) {  
  if (!tasksList || tasksList.length === 0) {  
    return null;  
  }  
  
  return (  
    <div className="col-5">  
      <h1>Доступные задачи:</h1>  
      <ul className="list-group list-group-flush">  
        {tasksList.map(item => (  
          <li key={item.task.ID} className="list-group-item">  
            <Link  
              to={`/${tasks}/${item.task.ID}`}  
              className="link-offset-2 link-offset-3-hover link-underline link-underline-  
opacity-0 link-underline-opacity-75-hover"  
            >  
              {item.task.name}  
            </Link>  
          </li>  
        ))}  
      </ul>  
    </div>  
  );  
}
```

ПРИЛОЖЕНИЕ Б

Фрагмент кода для отображения формы создания задачи

```
import React, { useState } from 'react';

function TaskForm() {
  const [formData, setFormData] = useState({
    title: "",
    description: "",
    urgency: 'medium', // low, medium, high
    deadline: "",
    department: ""
  });

  const departments = [
    'Отдел разработки',
    'Отдел маркетинга',
    'Отдел продаж',
    'Бухгалтерия',
    'Отдел поддержки',
    'HR'
  ];

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  };
}
```

```
    }));  
};
```

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  console.log('Отправленные данные:', formData);  
  // Здесь можно отправить данные на сервер (например, через fetch или axios)  
  alert('Задача создана:\n' + JSON.stringify(formData, null, 2));  
};
```

```
return (  
  <form onSubmit={handleSubmit} style={{ maxWidth: '600px', margin: '20px  
auto', padding: '20px', border: '1px solid #ddd', borderRadius: '8px' }}>  
    <h2>Создать новую задачу</h2>  
  
    <div className="mb-3">  
      <label htmlFor="title" className="form-label">Название</label>  
      <input  
        type="text"  
        id="title"  
        name="title"  
        className="form-control"  
        value={formData.title}  
        onChange={handleChange}  
        required  
      />  
    </div>
```

```
<div className="mb-3">
  <label htmlFor="description" className="form-label">Описание</label>
  <textarea
    id="description"
    name="description"
    className="form-control"
    rows="4"
    value={formData.description}
    onChange={handleChange}
    required
  />
</div>
```

```
<div className="mb-3">
  <label className="form-label">Срочность</label>
  <div>
    <div className="form-check form-check-inline">
      <input
        type="radio"
        id="urgencyLow"
        name="urgency"
        className="form-check-input"
        value="low"
        checked={formData.urgency === 'low'}
        onChange={handleChange}
      />
```

```

    <label htmlFor="urgencyLow" className="form-check-label">Низкая</label>
  </div>

  <div className="form-check form-check-inline">

    <input
      type="radio"
      id="urgencyMedium"
      name="urgency"
      className="form-check-input"
      value="medium"
      checked={formData.urgency === 'medium'}
      onChange={handleChange}
    />

    <label htmlFor="urgencyMedium" className="form-check-label">Средняя</label>
  </div>

  <div className="form-check form-check-inline">

    <input
      type="radio"
      id="urgencyHigh"
      name="urgency"
      className="form-check-input"
      value="high"
      checked={formData.urgency === 'high'}
      onChange={handleChange}
    />

    <label htmlFor="urgencyHigh" className="form-check-label">Высокая</label>
  </div>

```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className="mb-3">
```

```
  <label htmlFor="deadline" className="form-label">Сроки  
выполнения</label>
```

```
  <input
```

```
    type="date"
```

```
    id="deadline"
```

```
    name="deadline"
```

```
    className="form-control"
```

```
    value={formData.deadline}
```

```
    onChange={handleChange}
```

```
    required
```

```
</div>
```

```
<div className="mb-3">
```

```
  <label htmlFor="department" className="form-label">Исполняющий  
отдел</label>
```

```
  <select
```

```
    id="department"
```

```
    name="department"
```

```
    className="form-select"
```

```
    value={formData.department}
```

```
    onChange={handleChange}
```

```
    required
```

```

    >
    <option value="">Выберите отдел</option>
    {departments.map((dept, index) => (
      <option key={index} value={dept}>{dept}</option>
    ))}
  </select>
</div>

  <button type="submit" className="btn btn-primary">Создать
задачу</button>
</form>
);
}

export default TaskForm;

```


ПРИЛОЖЕНИЕ В

Фрагмент кода для отображения формы авторизации

```
import React, { useState } from 'react';
```

```
function LoginForm() {
```

```
  const [formData, setFormData] = useState({
```

```
    username: "",
```

```
    password: ""
```

```
  });
```

```
  const handleChange = (e) => {
```

```
    const { name, value } = e.target;
```

```
    setFormData(prev => ({
```

```
      ...prev,
```

```
      [name]: value
```

```
    }));
```

```
  };
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault();
```

```
    // Здесь обычно отправляют данные на сервер
```

```
    console.log('Данные для входа:', formData);
```

```
    alert(`Попытка входа:\nЛогин: ${formData.username}\nПароль: ${formData.password}`);
```

```
    // В реальном приложении: вызов API через fetch или axios
```

```
  };
```

```

return (
  <div style={{ maxWidth: '400px', margin: '50px auto', padding: '20px', border:
'1px solid #ddd', borderRadius: '8px' }}>
    <h2 className="text-center mb-4">Авторизация</h2>
    <form onSubmit={handleSubmit}>
      <div className="mb-3">
        <label htmlFor="username" className="form-label">Логин</label>
        <input
          type="text"
          id="username"
          name="username"
          className="form-control"
          value={formData.username}
          onChange={handleChange}
          required
        />
      </div>

      <div className="mb-3">
        <label htmlFor="password" className="form-label">Пароль</label>
        <input
          type="password"
          id="password"
          name="password"
          className="form-control"
          value={formData.password}
          onChange={handleChange}

```

```
        required
      />
    </div>

    <button type="submit" className="btn btn-primary w-100">Войти</button>
  </form>
</div>
);
}

export default LoginForm;
```